

Gridification de Nuages de Points Comparaison avec l'Optimal Transport

Armand Decacqueray

Mai 2026

1 Introduction

On considère un ensemble de points

$$\{(x_n, y_n)\}_{n=1}^N.$$

L'objectif est de construire une bijection

$$G : \{1, \dots, I\} \times \{1, \dots, J\} \rightarrow \{1, \dots, N\},$$

où $N = IJ$, permettant d'associer chaque point à une cellule d'une grille régulière tout en préservant au mieux la structure géométrique locale du nuage de points.

Autrement dit, on cherche une *gridification* du nuage.

Un critère naturel consiste à préserver les voisinages locaux. Par exemple :

$$\max_{m \in \text{KNN}(n, k)} \|G^{-1}(m) - G^{-1}(n)\|_{\infty},$$

où $\text{KNN}(n, k)$ désigne les k plus proches voisins du point n .

2 Algorithme de tri cartésien

Par simplicité, l'algorithme est présenté en dimension 2, mais il se généralise directement à des dimensions supérieures.

On note

$$x_g(i, j) = x_{G(i, j)}, \quad y_g(i, j) = y_{G(i, j)}.$$

Le critère cartésien recherché est :

- $x_g(i, j)$ croissant selon i ,
- $y_g(i, j)$ croissant selon j .

2.1 Méthode de base

L'algorithme itère les opérations suivantes jusqu'à satisfaction du critère précédent :

1. Pour chaque colonne j , trier les indices selon $x_g(:, j)$.
2. Pour chaque ligne i , trier les indices selon $y_g(i, :)$.

Autrement dit :

$$\begin{aligned} \sigma_j &\leftarrow \text{argsort}_i(x_g(i, j)), \\ x_g(i, j) &\leftarrow x_g(\sigma_j(i), j), \\ y_g(i, j) &\leftarrow y_g(\sigma_j(i), j), \end{aligned}$$

Ensuite :

$$\begin{aligned}\sigma_i &\leftarrow \text{argsort}_j(y_g(i, j)), \\ x_g(i, j) &\leftarrow x_g(i, \sigma_i(j)), \\ y_g(i, j) &\leftarrow y_g(i, \sigma_i(j)),\end{aligned}$$

L'algorithme converge systématiquement vers une grille vérifiant le critère cartésien.

2.2 Raffinement isotrope

Le critère précédent est relativement faible et possède de nombreux minima locaux.

En pratique, la méthode de base produit souvent des "axes faibles" : la qualité est excellente selon une direction mais nettement moins bonne selon l'autre.

Pour corriger ce biais anisotrope, on partitionne la grille en sous-grilles homogènes.

La partition "idéale" serait de faire 4 groupes selon la parité :

$$(\text{pair}, \text{pair}), (\text{pair}, \text{impair}), (\text{impair}, \text{pair}), (\text{impair}, \text{impair}).$$

À chaque itération :

1. deux sous-grilles sont triées selon i ,
2. les deux autres selon j ,
3. une nouvelle partition en sous grilles homogène est générée (on ne peut PAS garder la partition pair/impair parce que sinon on se retrouve avec 4 sous grilles decouplées a la fin)

Cette approche produit un comportement beaucoup plus isotrope.

2.3 Raffinement par cisaillement

Même avec le raffinement précédent, certaines lignes ou colonnes peuvent même consécutives peuvent être légèrement décorréliées (stratification), mais la moindre petite décoration affecte sensiblement la qualité du résultat.

Pour limiter la stratification, on introduit une méthode de couplage : il suffit tout simplement de légèrement *shuffle* à chaque itération -> on fini sur un minima beaucoup plus stable.

Le meilleur shuffle est obtenu par "cisaillement" :

$$\begin{aligned}(i, j) &\mapsto (i, j + i \bmod 2), = \text{cisaillement par ligne} \\ (i, j) &\mapsto (i + j \bmod 2, j) = \text{cisaillement par colonne}.\end{aligned}$$

Ce couplage améliore fortement la stabilité de la solution finale, mais l'algorithme converge beaucoup moins vite.

3 Complexité empirique

3.1 100 000 points

Méthode	Itérations	Temps
base	< 30	instantané
isotrope	~ 100	quelques secondes
shuffle	~ 1000	une minute

3.2 1 million de points

Méthode	Itérations	Temps
base	< 30	instantané
isotrope	~ 300	10/20 secondes
shuffle	~ 10 000	~ 20 minutes

4 Comparaison avec l'Optimal Transport

Plusieurs approches d'Optimal Transport ont été testées :

- **Linear Sum Assignment** : impraticable au-delà de quelques centaines de points ;
- **POT (Python Optimal Transport)** :
 - 2500 points : ~ 2 s,
 - 5000 points : ~ 12 s,
 - 10 000 points : ~ 55 s.
- plus que 10 000 : ralentissement massif (déjà rien que stocker le plan de transport devient compliqué)

note : Sinkhorn serait apparemment (mais en pratique j'ai trouvé le contraire??) plus rapide que OT exact, mais ça ne donne pas une vraie permutation

L'algorithme proposé est donc significativement plus scalable pour les très grands nuages de points (puisque je tire vraiment parti de la géométrie de la grille alors que OT résout un problème plus général).

De façon intéressante, la méthode "shuffle" et Optimal Transport produisent **quasiment le même transport** et obtiennent le même KNN score. Testé sur 10 000 points, on a peine à distinguer la différence entre OT et shuffle (voir images suivantes OT Versus SquareNet)





